

PROCESS DOCUMENTATION

Work About Work

How I Built a Documented AI-Assisted Diagnostic Tool in 10 Hours

Document type	AI Workflow Methodology and Process Record
Author	Ales
Total time	Approximately 10 hours from concept to live deployed tool
Approach	Documentation-first, human-in-the-loop AI product development
Output	Live web diagnostic tool + 5 product documents + test record

The Core Principle

This document records how I used AI to turn an existing research model into a live, fully documented web tool. It is not a record of what AI built. It is a record of how I structured the process so that AI could build reliably, traceably, and correctly.

"AI's default is to produce output immediately. The most important decision in this project was to say: not yet. Document first. Build second."

That single decision changed the nature of the output. Everything AI built in this project is traceable to a document I wrote and controlled. If any output was wrong, I had a source of truth to correct against. That is what a human-in-the-loop system looks like in practice.

01

The Starting Point

This project did not start from scratch. It started from three years of accumulated operational thinking:

Asset	What it contained	Role in this project
-------	-------------------	----------------------

Published article	The intellectual framework: work about work as a process defect, DMAIC methodology, 16.4-hour benchmark, research sources	Source of truth for all claims. Nothing in the tool contradicts the article.
Tableau model	Interactive ROI calculator with four adjustable inputs and live outputs across three admin categories	Calculation baseline. All formula outputs in the web tool were verified against this model during testing.
Research sources	Microsoft 365 telemetry, APQC benchmarks (n=982), Noy and Zhang (MIT), Harvard Business School	Grounded every assumption in the documentation. No numbers invented.

The goal was to make this existing thinking accessible to anyone in 60 seconds, without a Tableau login, without a spreadsheet, and without explanation.

02

The Decision: Document First, Build Second

The first instinct when working with AI on a build task is to start building immediately. The AI is ready. It will produce something fast. That instinct is wrong for anything that needs to be defensible, maintainable, or correct.

My intervention at the start of this project was to stop that instinct and ask a different question: what do I need to have decided and documented before AI builds anything?

"If any output is wrong, what do I correct it against? The answer to that question is the documentation set."

This is standard ops thinking applied to AI development. In any process design project, you define the requirements before you build the system. The documentation layer is the requirements. AI is the build tool.

Without documentation first	With documentation first
AI builds based on its best interpretation of the request	AI builds from an explicit spec with every decision already made
Errors are caught by looking at the output and guessing what went wrong	Errors are caught by comparing output to the source document

Changing one thing breaks other things unpredictably	Changes are applied to the document first, then reflected in the build
The process is not repeatable or transferable	Any developer can rebuild from the documents without asking questions

03

The Documentation Layer: Five Documents Before Any Code

Before a single line of HTML was written, five product documents were produced. Each was a deliberate decision about what the tool would do, how it would calculate, what it would ask, and what it would recommend.

#	Document	What I decided	What AI drafted
01	Framework Document	The three-part defect definition (measurable, systemic, avoidable). The DMAIC structure. The boundaries of the claim.	The full document structure, section copy, and supporting tables from my inputs and the published article.
02	Calculator Methodology Sheet	The reclaim coefficient (0.60). Fixed constants (48 weeks, 40 hours). What to expose as sliders vs lock in.	The full formula chain documentation, worked example, and stress test responses.
03	Diagnostic Question Bank	The five diagnostic dimensions. The symptom-to-cause sequence. The scoring bands (0-4 / 5-9 / 10-15). Total score drives output, not per-question routing.	The question text, option wording, scoring rationale, and sequence logic documentation.
04	Recommendation Logic Map	Validated AI-suggested recommendations against published research. Decided final form and ordering logic for each of the nine recommendations.	Draft recommendations grounded in published sources. I checked each source and confirmed the recommendation logic before approving.
05	Input/Output Specification	Currency (EUR). All slider ranges and defaults. Colour semantics. Layout rules. Number formatting rules.	The complete developer and designer brief from my decisions. This became the primary build brief for the HTML tool.

Total time for documentation layer: approximately 4 hours. This is not overhead. This is the work.

04

The Build: AI as Implementation Tool

With five product documents complete, the build brief was unambiguous. The I/O Specification contained every variable name, range, default, formula reference, colour value, display format, and layout rule. AI had no decisions left to make: only implementation.

Build element	Source document	AI role
Four calculator sliders	I/O Specification: Section 04	Implemented exactly as specified: variable names, ranges, defaults, display formats
Seven-step formula chain	Calculator Methodology Sheet: Section 05	Implemented in order of dependency with real-time oninput updates
Four metric outputs + two bars	I/O Specification: Section 05	Colour, position, format, and interpretation all specified in advance
Five audit questions	Question Bank: Section 04	Question text, option wording, and scoring taken directly from the document
Nine recommendations	Recommendation Logic Map: Sections 03-05	All nine recommendations reproduced exactly from the approved document
Brand and layout	I/O Specification: Sections 06-07	Colours, typography, two-panel layout, and number formatting applied from spec

Build time: approximately 2 hours including first prototype on Netlify.

05

The Test: Structured QA Against Source Documents

Testing was structured as a formal QA process, not an informal review. A 60-item test plan was produced before testing began, organised into ten sections. Every test had a defined expected result.

Test category	Method	Human role
Calculation verification	Manual calculation of all seven formula steps at specific inputs, then compared to live tool output	Set the test inputs. Ran the manual calculation. Identified the match or discrepancy.

Cross-model validation	Same inputs run in Tableau model and web tool, outputs compared	Identified and explained the coefficient difference between the two models. Documented it for the README.
Visual and brand QA	Screenshot review of all colour usage, typography, layout, and number formatting against I/O Specification	Caught colour logic inconsistency (magenta used for both brand and warning signal). Identified amber as off-brand. Proposed fixes.
Audit logic verification	All three score bands tested by selecting specific answer combinations	Verified correct band triggering and correct recommendation set for each band.
Content accuracy	Question text and recommendation copy checked against Question Bank and Recommendation Logic Map	Confirmed all content matches the approved source documents exactly.

06

The Change Log: Human Decisions That Improved the Output

Testing produced six confirmed changes. Each change originated from a human observation, not an AI suggestion. Each was logged before being applied, and each was reflected back into both the HTML file and the relevant source documents.

#	Change	Human observation	Ops reasoning
001	Admin load slider: % to hours per week	Percentages are abstract. People know their calendar, not their percentage.	Hours are more intuitive and more honest as a user input. Percentage conversion happens internally.
002	Hours per person per week: body text to magenta	Colour logic was inconsistent: some cost metrics were magenta, one was black.	Semantic colour system: magenta = cost/problem, green = opportunity. No exceptions.
003	Admin severity bar: amber removed, two colours only	Amber is not in the brand palette and introduces a third signal with no clear meaning.	Simplify to binary: green (manageable) or magenta (significant). Cleaner and more decisive.
005	README: note on Tableau vs web tool coefficient difference	Cross-model testing revealed different reclaim outputs for the same inputs.	The difference is methodological, not an error. It must be documented transparently.

00 6	Em dashes removed from all UI copy and documents	Em dashes create an editorial tone that does not fit a professional diagnostic tool.	Replaced with colons, commas, or restructured sentences throughout.
00 7	Score band banners: coloured backgrounds replaced with left border design	Three different background colours created visual inconsistency across the three bands.	Neutral background with coloured left border and title: cleaner, more professional, easier to extend.

Each change was applied to the HTML file and reflected back into the relevant source document. The documentation set and the tool are always in sync.

07

The Human/AI Division of Labour

The following table summarises who did what across the full project. The distinction matters: AI produced volume and speed. Human judgment produced correctness, coherence, and credibility.

Activity	Human	AI
Original research and intellectual framework	Wrote the article. Developed the DMAIC framing. Built the Tableau model.	None.
Decision to document before building	Stopped the build instinct. Defined the documentation set required.	None.
Key content decisions in each document	Scoring bands. Reclaim coefficient justification. Slider ranges. Currency. Colour semantics. Question sequence logic.	Proposed options and structured the decisions. Did not make them.
Recommendation content	Brainstormed with AI. Reviewed each recommendation against cited sources. Decided final form.	Drafted recommendations grounded in published research. Surfaced sources for human review.
Document drafting	Reviewed, corrected, and approved each document before it was used as a build input.	Drafted all five product documents from human-defined inputs and decisions.
HTML build	Provided the brief (I/O Specification). Reviewed the output.	Built the complete two-mode HTML tool from the specification.

Testing and QA	Ran all tests. Caught all six issues. Wrote the change log. Made every change decision.	Provided the test plan structure. Applied approved changes.
Cross-model validation	Ran the same inputs in Tableau and the web tool. Identified and explained the difference.	Performed manual calculation to verify formula chain.

08

What This Process Produces That Vibe Coding Does Not

"Vibe coding" describes the practice of prompting AI to build something and iterating on the output until it looks right. It is fast. It is also fragile: the output has no documented foundation, no traceable decisions, and no reliable way to correct errors or extend the system.

This project produced something different:

Property	Vibe coding approach	This approach
Correctability	Fix by prompting until it looks right	Fix by comparing to the source document and applying a documented change
Transferability	Only the original prompter can work on it	Any developer can rebuild from the five documents without asking questions
Credibility	Numbers and claims are AI-generated with no traceable source	Every number traces back to a published benchmark or peer-reviewed study
Extensibility	Adding a feature risks breaking existing behaviour unpredictably	Adding a feature means updating the relevant document first, then the build
Auditability	No record of what was decided or why	Full decision record across five documents plus a six-item change log

09

The Output: What Was Produced in 10 Hours

From concept to live deployed tool, the total elapsed time was approximately 10 hours. The output was not just a tool.

Deliverable	Description
-------------	-------------

Framework Document	Intellectual foundation grounding all claims in published research
Calculator Methodology Sheet	Full formula chain with assumptions, constants, and stress test responses
Diagnostic Question Bank	Five questions with full scoring rationale and sequence logic
Recommendation Logic Map	Nine recommendations with ordering logic and source citations
Input/Output Specification	Complete developer and designer brief with all variables, formats, and layout rules
Test Plan	60-item structured QA checklist across 10 test categories
Change Log	Six documented changes with human observation and ops reasoning for each
Live HTML tool	Two-mode diagnostic tool: calculator and audit, deployed on Netlify and GitHub Pages
Session Handoff Document	Complete project summary enabling any future session to continue without context loss

"The goal was not to build fast. It was to build something that could be trusted, corrected, and extended. Documentation-first AI development is how you get there."

10

The Replicable Pattern

This process is not specific to this tool. It is a replicable pattern for any AI-assisted product or process build where correctness, credibility, and maintainability matter.

Step 1: Start with what you already know

Existing research, models, or frameworks are your foundation. AI builds on what you bring, not from nothing.

Step 2: Define the documentation set before building

Ask: if the output is wrong, what do I correct it against? The answer defines the documents you need.

Step 3: Make all key decisions in the documents

Every decision made in a document is a decision AI does not get to make during the build. That is quality control.

Step 4: Use AI to draft, not to decide

AI drafts efficiently from your decisions. You review, correct, and approve before anything becomes a build input.

Step 5: For recommendations: brainstorm with AI, validate against sources, decide yourself

AI surfaces options and grounds them in research. Human judgment decides the final form.

Step 6: Build from the spec, not from a description

The more precise the build brief, the less AI has to interpret. Interpretation is where errors enter.

Step 7: Test against the source documents

Every test compares output to a document, not to a feeling about what looks right.

Step 8: Log changes and apply them back to the documents

The documentation set and the tool must always be in sync. Changes to one require changes to the other.